

PROGRAMMATION FONCTIONNELLE 2 : TD1

janvier 2026 — Pierre Rousselin

On commence par travailler avec les booléens. En Rocq, on considère

```
Inductive bool := true | false.
Definition andb (b1 b2 : bool) : bool :=
  match b1 with
  | true => b2 (* andb_true_l *)
  | false => false (* andb_false_l *)
  end.
Definition negb (b : bool) : bool :=
  match b with
  | true => false (* negb_true *)
  | false => true (* negb_false *)
  end.
```

Pour se simplifier la vie, on considère aussi les notations :

```
Notation "b1 && b2" := (andb b1 b2).
Notation "! b" := (negb b).
```

Exercice 1 :

1. En utilisant les noms donnés en commentaire pour chaque cas de `match` pour justifier chaque égalité, réduire au maximum le terme

$$(!(\text{true} \ \&\& \ \text{true})) \ \&\& \ (!\text{false} \ \&\& \ \text{true})$$

2. Réécrire les définitions de `andb` et `negb` en utilisant `if/then/else`.

3. Donner la définition en Rocq de l'opération « ou » booléenne `orb`.

4. Pour se simplifier la vie, on se donne la notation :

```
Notation "b1 || b2" := (orb b1 b2).
```

Évaluer en justifiant chaque égalité

$$(\text{true} \ || \ \text{false}) \ \&\& \ (\text{true} \ || \ \text{true})$$

5. Prouver sur papier avec des preuves par cas :

- a) Pour tout booléen b , $!(!b) = b$ (! est involutive).
- b) Pour tous booléens b_1, b_2, b_3 , $b_1 \ \&\& \ (b_2 \ \&\& \ b_3) = (b_1 \ \&\& \ b_2) \ \&\& \ b_3$ (associativité de $\&\&$).
- c) Pour tous booléens b_1, b_2 , $b_1 \ \&\& \ b_2 = b_2 \ \&\& \ b_1$ (commutativité de $\&\&$).
- d) Pour tous booléens b_1, b_2, b_3 , $b_1 \ \&\& \ (b_2 \ || \ b_3) = (b_1 \ \&\& \ b_2) \ || \ (b_1 \ \&\& \ b_3)$ (distributivité à gauche de $\&\&$ sur $||$).
- e) Pour tout booléen b , $b \ || \ !b = \text{true}$ (tiers-exclu pour les booléens).

--- * ---

Correction de l'exercice 1 :

1. Il y a plusieurs stratégies possibles.

$$\begin{aligned} &!(\text{true} \ \&\& \ \text{true}) \ \&\& \ (!\text{false} \ \&\& \ \text{true}) \stackrel{\text{andb_true_l}}{=} !(\text{true}) \ \&\& \ (!\text{false} \ \&\& \ \text{true}) \\ &\stackrel{\text{negb_true}}{=} \text{false} \ \&\& \ (!\text{false} \ \&\& \ \text{true}) \\ &\stackrel{\text{andb_false_l}}{=} \text{false} \end{aligned}$$

2. **Definition** `negb (b : bool) := if b then false else true.`
Definition `andb (b1 b2 : bool) := if b1 then b2 else false.`
3. **Definition** `orb (b1 b2 : bool) := if b1 then true else b2.`
- 4.

$$\begin{aligned}
 (\text{true} \parallel \text{false}) \&\& (\text{true} \parallel \text{true}) && \overset{\text{orb_true_1}}{=} \text{true} \&\& (\text{true} \parallel \text{true}) \\
 && \overset{\text{andb_true_1}}{=} \text{true} \parallel \text{true} \\
 && \overset{\text{orb_true_1}}{=} \text{true}
 \end{aligned}$$

- 5.a) Soit b un booléen. On raisonne par cas. Si $b = \text{true}$, alors

$$!(b) = !(\overset{\text{negb_true}}{!} \text{true}) \overset{\text{negb_false}}{=} \text{false} \text{ true} = b.$$

Si $b = \text{false}$, alors

$$!(b) = !(\overset{\text{negb_false}}{!} \text{false}) \overset{\text{negb_true}}{=} \text{true} \text{ false} = b.$$

--- * ---

Exercice 2 : Eval simpl à la main

La commande Rocq

`Eval simpl in expr.`

tente de réduire au maximum `expr` en calculant à l'aide des définitions. On suppose donnée une variable `b` type `bool`¹. Dans chaque cas suivant, dire ce qu'affiche la commande `Eval simpl`.

Section `ExerciceEvalSimplTD1.`

Context `(b : bool).`

`(* 1 *) Eval simpl in (true && b).`

`(* 2 *) Eval simpl in (b && true).`

`(* 3 *) Eval simpl in (false && b).`

`(* 4 *) Eval simpl in (b && false).`

`(* 5 *) Eval simpl in (b && (!false)).`

`(* 6 *) Eval simpl in (!(!false) || b).`

End `ExerciceEvalSimplTD1.`

--- * ---

Correction de l'exercice 2 :

1. `= b : bool`
2. `= b && true : bool`
3. `= false : bool`
4. `= b && false : bool`
5. `= b && true : bool`
6. `= b : bool`

--- * ---

Exercice 3 : Autres opérations

Donner deux définitions possibles en Rocq de chacune des opérations booléennes suivantes, l'une avec `match`, l'autre avec des opérations déjà définies.

1. pour les curieux, ceci est fait avec la commande `Context` disponible dans une `Section`, comme écrit ci-dessous

1. la fonction « ou exclusif », `xorb`
2. l'implication, `implb`
3. l'égalité booléenne des booléens, `eqb`

--- * ---

Correction de l'exercice 3 :

```

Definition xorb (b1 b2 : bool) :=
  match b1 with
  | true => negb b2
  | false => b2
  end.
Definition implb (b1 b2 : bool) :=
  match b1 with
  | false => true
  | true => b2
  end.
Definition eqb (b1 b2 : bool) :=
  match b1, b2 with
  | true, true | false, false => true
  | _, _ => false
  end.
Definition xorb' (b1 b2 : bool) :=
  (b1 || b2) && !(b1 && b2).
Definition implb' (b1 b2 : bool) :=
  b2 || !b1.
Definition eqb' (b1 b2 : bool) :=
  (b1 && b2) || (!b1 && !b2).
(* de nombreuses autres possibilités ... *)

```

--- * ---

Exercice 4 : Logique ternaire

On considère dans cet exercice une logique à trois valeurs : vrai, faux et « inconnu ».

1. Écrire la définition en Rocq d'un type `ternary` à trois constructeurs constants qui sont `truet`, `false` et `unknown`.
2. Définir avec `match` (mais pas trop de cas) les opérations `negt`, `ort` et `andt` pour la négation, le « ou logique » et le « et logique » pour cette logique avec inconnu.
3. La négation est-elle encore involutive ? Les opérations `ort` et `andt` sont-elles encore commutatives ? associatives ?

--- * ---

Correction de l'exercice 4 :

```

Inductive ternary := truet | false | unknown.
Definition negt (t1 t2 : ternary) :=
  match t1 with
  | truet => false
  | false => truet
  | unknown => unknown
  end.
Definition andt (t1 t2 : ternary) :=
  match t1 with
  | truet => t2

```

```
| falset => falset
| unknownt =>
  match t2 with
  | falset => falset
  | _ => unknownt
  end
end.
Definition ort (t1 t2 : ternary) :=
  match t1 with
  | truet => truet
  | falset => t2
  | unknownt =>
    match t2 with
    | truet => truet
    | _ => unknownt
    end
  end
end.
```

On peut se convaincre (mais ne pas traiter formellement tous les cas) que `ort` et `andt` sont encore commutatives et associatives et que `negt` est encore involutive. On fera les preuves en TP.

--- * ---